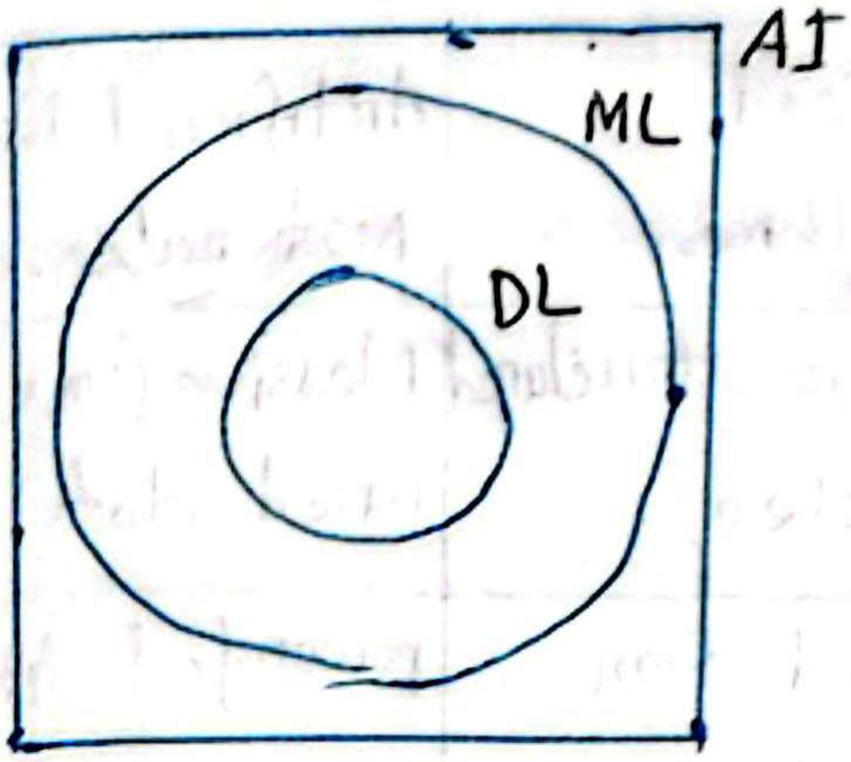
Date : 20-01-26



AI: The broad field of creating machines that mimic human intelligence and reasoning.

ML: A subset of AI that uses algorithms to learn patterns from data without being explicity programmed.

DL: A subset of ML that uses multi-layered neural networks to solve complex task like image or speech recognition.

Data scientist def:

A data scientist is a professional who uses statistics, programming, and machine learning to turn raw data into actionable insights and future predictions.

* Differences between AI, ML and DL:

Feature	AI	ML	DL
Definition	The broad science of mimicking human intelligence.	A subset of AI that learns patterns ~~and~~ from data.	A subset of ML using multi layered neural networks.

Core method	Logic, rules or algorithms.	Statistical algorithms.	Artificial Neural Networks.
Data need	Varies (can be zero data)	High (pre structured or labeled).	Massive (unstructured data)
Hardware	Standard computers.	CPUs and some memory.	powerful GPUs or TPUs.
Complexity	Simple to very complex	Medium complexity	Highest complexity.
Example	A chess playing program.	A netflix recommendation.	A self driving car system.

* Definition of Reinforcement, supervised, and unsupervised learning.

⇒ Supervised learning:

Supervised learning is a type of machine learning where the model is trained on labeled data.

Two main types:

Classification: is a supervised learning task where the goal is to predict a discrete label or category for a given input.

Regression: is a supervised learning task where the goal is to predict a continous numerical value or quantity.

Unsupervised learning: is a type of machine learning where the model is trained on unlabeled data without any "answer key" or teacher.

Reinforcement learning: is a type of Machine Learning where an agent learns to make decisions by performing actions in an environment to ~~achieve~~ achieve a goal.

~~*Definition of ANN, CNN, RNN with example and where used?~~

~~⇒ ANN: is a machine learning model inspired by the human brain that learns patterns from data using interconnected neurons. It's Here, tabular data used.~~

~~Example: input: Study hours, attendance~~
~~output: pass/fail prediction.~~

~~Where used:~~ (i) ~~Image recognisition~~
(ii) ~~Speech and text prediction processing.~~
(iii) ~~Medical diagnosis.~~
(iv) ~~Fraud detection~~
(v) ~~Recommendation systems.~~

~~Example: Email spam filter.~~

~~The input: The words and sender details of an email.~~

~~process: The ANN assigns weights to different words.~~

~~Words like "Winner" or "free" increase the probability of it being spam~~

~~Output: A probability score (98% spam). If its high, the email goes to the junk folder~~

~~Where it is used:~~

~~① Image recognition: Identifying faces in photos.~~

~~② Finance: Predicting stock market trends or detecting credit card fraud.~~

~~③ Healthcare: Analyzing X-rays and MRIs to detect tumors.~~

~~④ Natural Language: Powering translation tools like Google translate.~~

* Definition of ANN, CNN, RNN with example and where to use.

⇒ ANN:

Def: The most basic type of neural network consisting of an input layer, one or more hidden layers, and an output layer. It is "feed-forward", meaning data flows only in one direction.

Example: A system that takes customer details (age, income, credit score) and predicts if they will default on a loan.

Where to use:

① Tabular/structured data: Best for data organized in rows and columns, like Excel sheets.

② Simple classification: Email spam detection or credit score.

③ Regression: Predicting house prices based on features like area and location.

CNN:

Def: A specialized network designed to process grid-like data (images) using "filters" to automatically detect spatial features like edges, shapes and textures.

Example: An app that identifies a plant's species based on a photo you upload.

Where to use:

(i) Computer vision: self-driving cars

(ii) Healthcare: Analyzing X-rays or MRIs to detect ~~dear~~ diseases.

(iii) Security: Scanning luggage at airports for restricted items.

RNN:

Definition: A network designed for sequential data. It has "memory" loops that allow it to use information from previous inputs to influence the current output.

Example: Siri or Alexa. These assistants must remember the beginning of your sentence to understand the context of the end of your sentence.

Where to use:

(i) ~~Neural~~ NLP: language translation and chatbots.

(ii) Time series forecasting: Predicting stock market prices or weather patterns.

(iii) Audio/Music/Speech-to-text conversion and music generation

* ANN vs CNN vs RNN:

	ANN	CNN	RNN
Best for	General purpose tasks.	Image and spatial data.	Sequential / time series data.
Data type	Structured/ tabular data.	Images/videos	Text, speech, time series.
Core idea	Fully connected layers.	Convolution/pooling layers.	Feedback loops (memory)
Memory on past input	No	No	Yes
Example to use case	Credit scoring	Face recognition	Language translation.

Perceptron: [Single layer network]

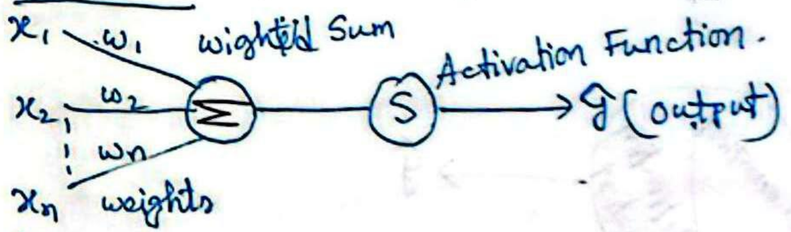


Fig: Perceptron

Def: A perceptron is the simplest type of artificial neuron. It takes inputs, multiplies them by weights, adds a bias and produces an output.

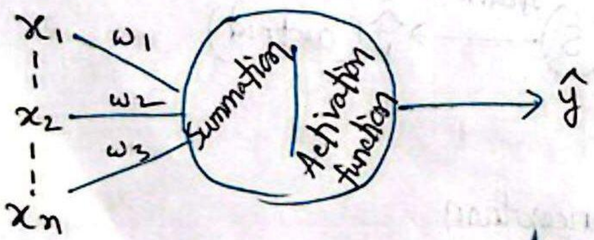
* **weight (w)**: A weight determines how important an input is. Each input has its own weight.
* **Bias (b)**: Bias is an additional constant added to the weighted sum. It helps shift the decision boundary.

Without bias, the model always pay through zero.

Forward Propagation:

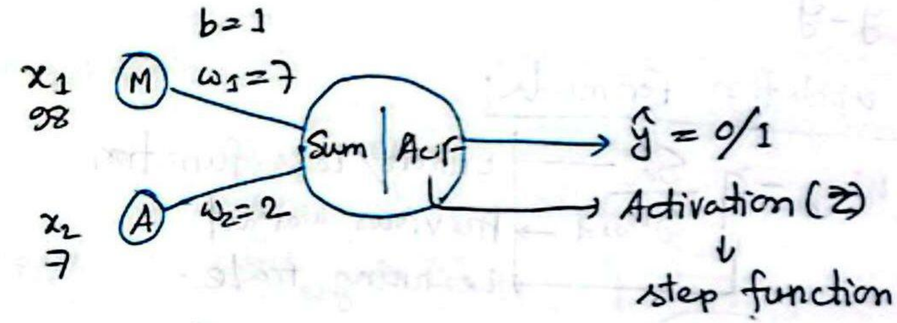
Is the process of:

1. Taking inputs.
2. Multiplying by weights.
3. Adding bias.
4. Passing through the activation function.
5. Producing output.



Marks	Attendance	Placement
98	7	1
30	2	0

Activation function bounds the value of sum on z into a certain range.



loss $= y - \hat{y}$

if, $\hat{y}=1$, loss $= 1-1=0$

if, $\hat{y}=0$, loss $= 1-0=1$

The value of weight depends on how important the feature is.

$$z = x_1 w_1 + x_2 w_2 + b1$$

$$= \sum_{i=1}^{n} x_i w_i + b$$

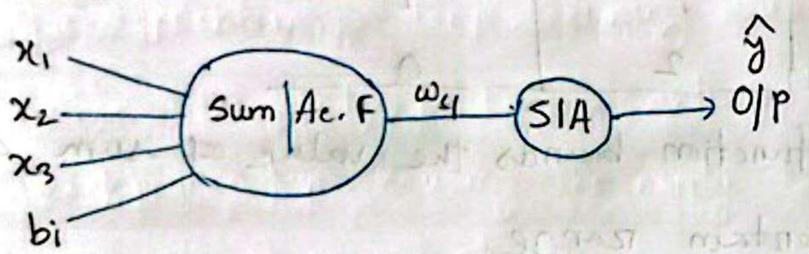
[To reduce the loss, start backward propagation

Perceptron works when data is linearly separable.

→ Suppose: w_1 ↓ A, w_2 ↓ B, x_1 ↓ X, x_2 ↓ Y, b ↓ C

$= AX + BY + C$ [line equation for 2D]

Forwarded Propagation:



Loss = $y - \hat{y}$

*Weight updation formula:

$$w_{new} = w_{old} - \eta \frac{\delta L}{\delta w_{old}}$$

$\frac{\delta L}{\delta w_{old}}$ → Error / Loss function
w_{old} → Previous weight
η → Learning rate.

$$w_{4_{new}} = w_{4_{old}} - \eta \left[\frac{\delta L}{\delta w_{4_{old}}}\right] \rightarrow \text{Slope. Slop.}$$

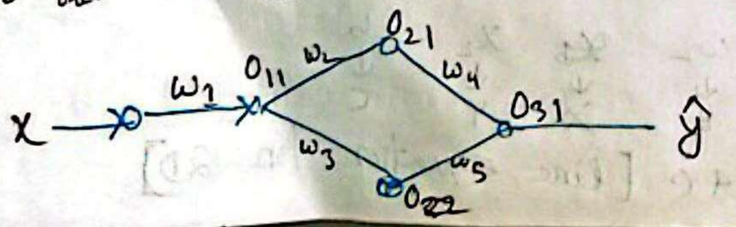
*Chain of Derivation:

$$\frac{\delta L}{\delta w_{4_{old}}} = \frac{\delta O_2}{\delta w_{4_{old}}} \times \frac{\delta L}{\delta O_2}$$

for w_2:

$$w_{2_{new}} = w_{2_{old}} - \eta \frac{\delta L}{\delta w_{2_{old}}}$$

$$\frac{\delta L}{\delta w_{2_{old}}} = \frac{\delta O_1}{\delta w_{2_{old}}} \times \frac{\delta O_2}{\delta O_1} \times \frac{\delta L}{\delta O_2}$$



$$O_{11} = z(x w_1 + b)$$

$$O_{21} = z(O_{11} w_2 + b)$$

$$O_{22} = z(O_{11} w_3 + b)$$

$$O_{31} = z(O_{11} w_4 + O_{11} w_5 + b)$$

*Weight update formula:

*update w_1

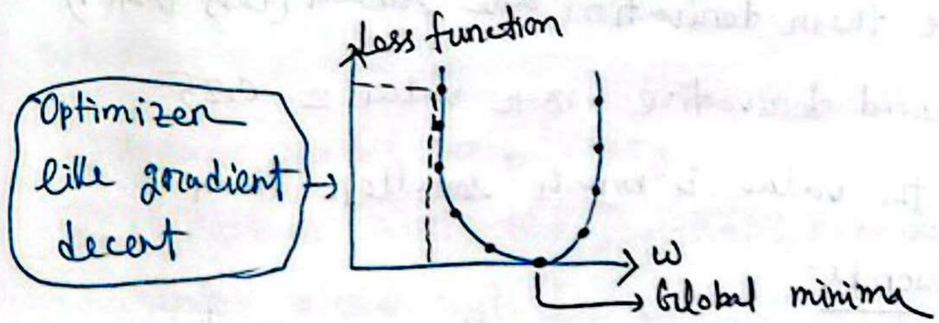
$$\rightarrow w_{new} = w_{old} - \eta \frac{\delta L}{\delta w_{old}}$$

update w_1, $w_{1_{new}} = w_{1_{old}} - \eta \frac{\delta L}{\delta w_{1_{old}}}$

$$\frac{\delta L}{\delta w_{1_{old}}} = \left[\frac{\delta O_{11}}{\delta w_{1_{old}}} \times \frac{\delta O_{21}}{\delta O_{11}} \times \frac{\delta O_{31}}{\delta O_{21}} \times \frac{\delta L}{\delta O_{31}}\right] + \left[\frac{\delta O_{11}}{\delta w_{1_{old}}} \times \frac{\delta O_{22}}{\delta O_{11}} \times \frac{\delta O_{31}}{\delta O_{22}} \times \frac{\delta L}{\delta O_{31}}\right]$$

if $w_{new} > w_{old}$ slop ↓ and -ve

if $w_{new} < w_{old}$ slop ↑ and +ve



Vanishing Gradient Decent Problem:

is a problem in deep learning neural networks where the gradients become extremely small during backpropagation, causing the early layers to learn very slowly or not at all.

Reasons:

1. During training we updates weights using gradients: $\frac{\delta L}{\delta w_{old}}$

   If the derivation are small numbers, multiplying many of them makes the result extremely small.

2. Ex: $0.1 \times 0.1 \times 0.1 \times 0.1 \times 0.1 = 0.00001 \approx 0$

   weight becomes very slow = learning stops.

2. Activation functions:

   For sigmoid and tanh.

   Because their derivation are small (Less than 1)

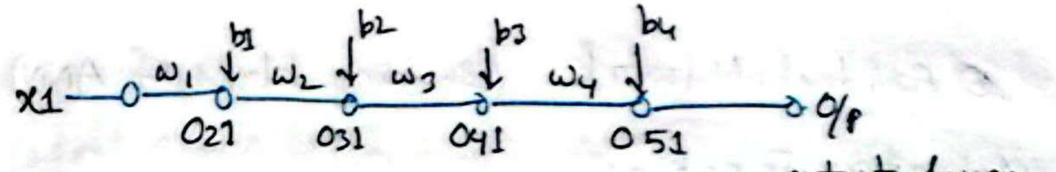
   Ex: Sigmoid derivative, max value = 0.25

   Most of the value is much smaller than 1.

3. Deep Networks:

   More multiplications: → smaller gradients

   → Other layer stop learning.



Sigmoid Function: $\delta(x) = \frac{1}{1+e^{-x}}$

$o51 = \delta(o41 \times w_4 + b_4)$

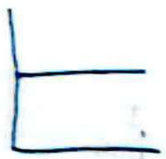
$w1_{new} = w1_{old} - \eta \frac{\delta L}{\delta w1_{old}}$

$\frac{\delta L}{\delta w1_{old}} = \frac{\delta o21}{\delta w1_{old}} \times \frac{\delta o31}{\delta o21} \times \frac{\delta o41}{\delta o31} \times \frac{\delta o51}{\delta o41} \times \frac{\delta L}{\delta o51}$

$w1_{new} \approx w1_{old}$

Recognization:

→ Focus loss – how much gap between the predict and Real value.

→ weight graph 

Solution:

① Reduce model complexity.

② Use other activation function: ~~ReLU~~ ReLU ($y = max(0, x)$)

③ Proper weight initial

④ Batch normalization.

⑤ Residual Network (Building blocks of ANN)

Activation Function:

In Ann each neuron forms a weighted sum of its inputs and pains the resulting scaler value through a function referred as an activation function. If a neuron has n inputs than the output on activation of a neuron is;

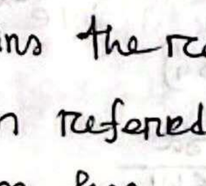
$$a = g(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$

characteristics:

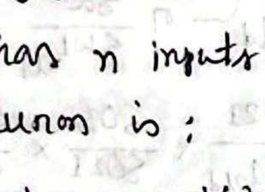
① Non linear
② Differentiable
③ Computational inexpensive
④ Zero centered → (data normalize - min = 0)
⑤ Non - saturing.

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$



Sigmoid is an activation function that converts input values into a range between 0 and 1 using an S-shaped curve.

*** Advantages:**

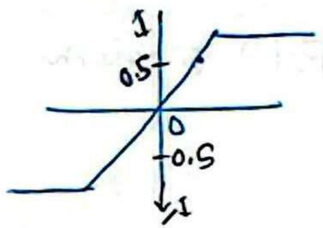
- Output range (0,1) good for probability.
- Smooth and differentiable
- Historically used in binary classification output layer.

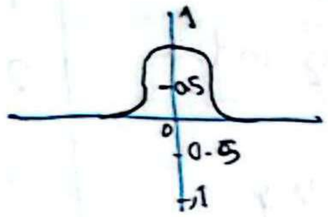
Disadvantages:

- Causes vanishing gradient
- Not-zero centered
- Slow convergence.

Tanh Activation Function:

Tanh is an activation function that maps input values to a range between -1 and 1 using a hyperbolic tangent function.





$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(x)' = (1 - \tanh^2(x))$$

Advantages

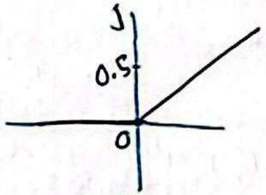
1. Non linear data
2. Differentiable
3. Zero centered

disadvantages

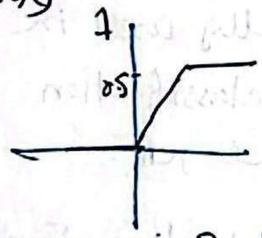
1. Computationally complex
2. Saturing function.

ReLU:

ReLU is an activation function that outputs 0 for negative inputs and the input value itself for positive inputs. $f(x) = \max(0, x)$



$$\text{Relu}(x) = \begin{cases} 0 ; x<0 \\ x ; x \geq 0 \end{cases}$$



$$\text{ReLU}'(x) = \begin{cases} 0 ; x<0 \\ 1 ; x \geq 0 \end{cases}$$

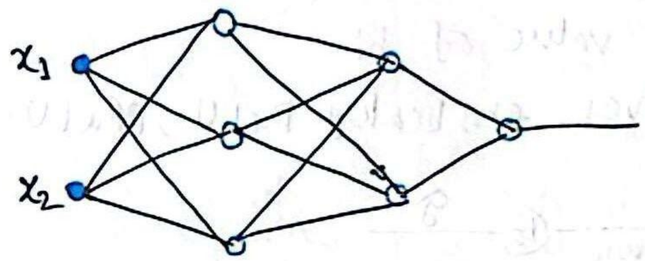
Advantages

1. Non linear
2. Not saturing in the region.
3. Computationally inexpensive.
4. Converge → faster than (tanh, Sigmoid)

Disadvantages

1. Non zero centered
2. Dying ReLU problem

Dying ReLU Problem:



$f(x) = \max(0, x)$

Definition: The dying ReLU problem refers to the scenario when many ReLU neurons only output values of 0. The red outline below shows that this happens when the inputs are in the negative range. $x=0$

* $W_n = W_0 - \eta \frac{\delta L}{\delta W_0}$
* η + high ↑
* b − high ↑
* high value of learning rate.

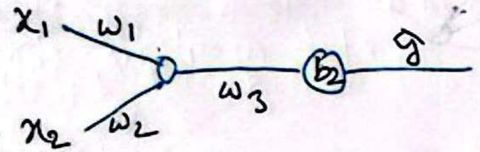
if our learning rate (η) is set too high, there is a significant ~~case~~ chance that our new weights will end up in the highly negative value range since of old weights will be substracted by a large number. These negative weights results in negative input for ReLU, there by causing the dying ReLU problem to happen.

Solution:

→ η small value.

→ Take positive value of b_i

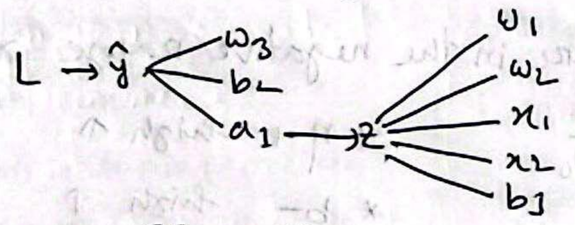
→ use alternatives. ex: Leaky ReLU, PReLU.



Sum $= z_1 = w_1 x_1 + w_2 x_2 + b_1$

ReLU, $x = \max(0, z) = 0$

* Loss depends on $\hat{y}$



$$\text{for, } w_1 = \frac{\delta L}{\delta w_1} = \frac{\delta z_1}{\delta w_1} \times \frac{\delta a_1}{\delta z_1} \times \frac{\delta \hat{y}}{\delta a_1} \times \frac{\delta L}{\delta \hat{y}}$$

$$\text{for, } w_2 = \frac{\delta L}{\delta w_2} = \frac{\delta z_1}{\delta w_2} \times \frac{\delta a_1}{\delta z_1} \times \frac{\delta \hat{y}}{\delta a_1} \times \frac{\delta L}{\delta \hat{y}}$$

↳ Neuron will be dead if some how this value become zero.

* $b_{new} = b_{old} - \eta \frac{\delta L}{\delta b_{old}}$

if this value is high then it will return negative value and the neuron will be dead.

Alternatives of ReLU

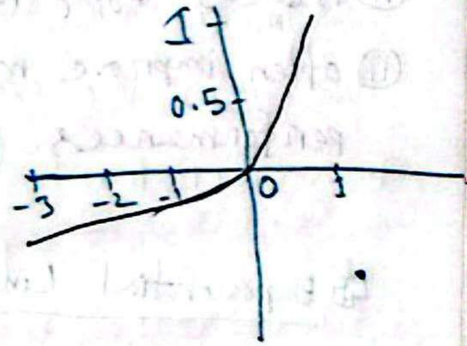
- Linear
  - Leaky ReLU
  - Parametric ReLU
- Non-Linear
  - ELU (Exponential Linear Unit)
  - SELU (Scaled Exponential Linear Unit)

Leaky ReLU:

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

α = is a small constant (eg. 0.01)

$$f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$



Advantages

(i) Reduce dying ReLU problem

(ii) Not saturating

(iii) zero centered

(iv) Easy to compute

disadvantages

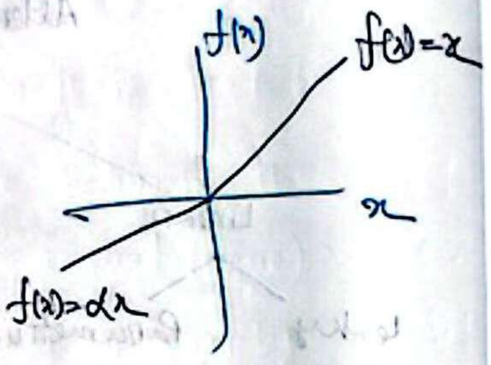
(i) Negative slope is fixed

(ii) Still not self-normalizing

PReLU (Parametric ReLU):

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

α is a learnable parameter depends on train data.



Advantages:

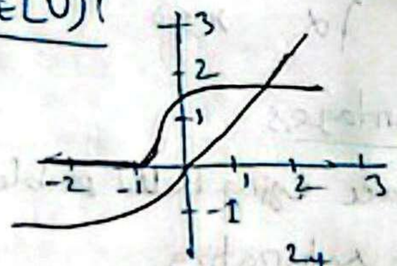
(i) More flexible than Leaky ReLU.
(ii) Reduces dying ReLU.
(iii) often improve model performances.

Disadvantages

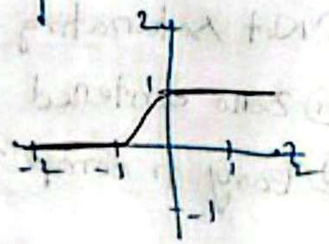
(i) Adds extra trainable parameter.
(ii) Risk of overfitting.
(iii) Slightly higher computation.

Exponential Linear Unit (ELU):

$$ELU(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$



$$ELU(x)' = \begin{cases} 1 & \text{if } x > 0 \\ ELU(x) + \alpha & \text{if } x \leq 0 \end{cases}$$



Advantages

(i) Close to zero centered
(ii) Reduces gradient descent 'Fasten learning

Disadvantages

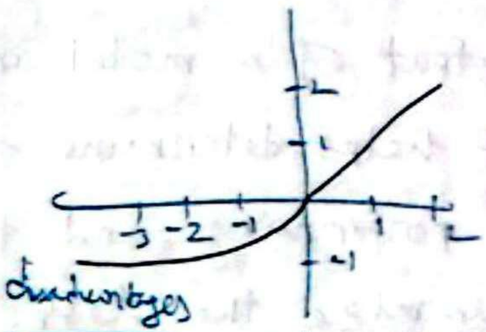
(i) Computationally expensive
(ii) requires parameter α

SELU (Scaled ELU):

$$f(x) = \lambda \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

where $\lambda \approx 1.0507$
$\alpha \approx 1.67326$

$$f'(x) = \begin{cases} \lambda & x > 0 \\ \lambda \alpha e^x & x \leq 0 \end{cases}$$



Advantages

(i) Self normalizing
(ii) Fasten training

Disadvantages

(i) Needs careful initializations.
(ii) special dropout required

Softmax Activation function:

Definition: Softmax is an activation function that converts multiple output values into probabilities whose total sum equals 1. mainly used for multi-class classification.

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

Advantages:

(i) Produces probability distribution.
(ii) Output values sum to 1.
(iii) Ideal for multi-class classification

disadvantages

(i) computationally expensive.
(ii) Sensitive to large input values.
(iii) Output are independent

Loss function:

Def: A loss function is a mathematical function that measures the difference between the predicted output of a model and the actual (true) value. It helps determine how well or poorly a model is performing, and the goal of training is to minimize the loss.

- Measures prediction error
- Guides weight updates during back propagation.
- Helps improve model accuracy.

Cost function:

Def: A cost function is the average of the loss values for all training examples in a dataset.

Loss function	Cost function
1. Single training example.	1. Entire dataset
2. Measures individual error.	2. Measures overall error.
3. Shows how wrong a single prediction is.	3. Evaluates the overall model performance.
4. Error for one sample.	4. Average/sum of losses.

Loss function.

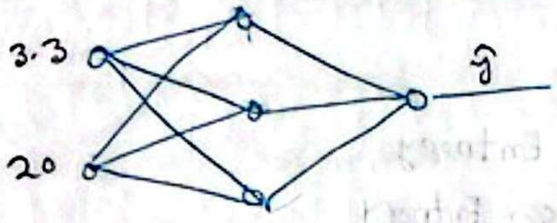
- Object Detection Loss
- Regression
  - MSE (Mean Squared Error)
  - MAE (Mean Absolute Error)
  - Huber loss
- Classification.
  - Binary Cross Entropy
  - Categorical Cross Entropy
- Auto encoder
  - KL divergence
- Embedding
  - Triplet loss
- GAN
  - Discriminitory loss
  - Min-Max GAN loss.

MSE (Mean Squared Error):

Def: Is a loss function that measures the average of the squared differences between the predicted values and the actual values. It is commonly used in regression models.

*Loss: $= (y_i - \hat{y}_i)^2$ *Cost: $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

CGPA	IQ	Salary (K)	y
3.3	20	2.5	1
3.1	10	2	0
3.8	100	10.5	12
3.7	70	7.3	6



Advantages

① Easy to compute and differentiate

② Smooth and continues.

③ Works well in regression

disadvantages

① Very sentitive to outliers

② Large errors dominate training.

③ Not suitable for classification

Mean Absolute Error (MAE)

Def: is a loss function that measures the average of the absolute differences the predicted values and the actual values. It is also commonly used in regression tasks.

loss function: $|y_i - \hat{y}_i|$

cost function: $= \frac{1}{n} \sum_{i=1}^{n} (|y_i - \hat{y}_i|)$

Advantages

① Less sensitive to outliers

② Easy to interpret

③ Good for regression

disadvantages

① Not differentiable at 0.

② Slower convergence than MSE.

Huber loss:

Def: Huber loss is a loss function that is quadratic for small errors and linear for large errors, reducing the effect of outliers.

$$L_\delta(y, \hat{y}) = \begin{cases} 1/2(y-\hat{y})^2 & \text{if } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

δ - hyper parameter.

*Small error – MSE

*Large Error – MAE

Advantages

① Differentiable everywhere

② Combine benefit of MSE and MAE

③ Good for robust regression.

disadvantages

① More complex than MAE/MSE

② Not suitable for classification.

③ Performance depends on proper δ tuning.

Binary class entropy:

Binary Cross Entropy is a loss function used for binary classification problems (two classes). It measures the differences between the predicted probability and the actual label (0 or 1).

$$\text{loss} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

$$\text{cost} = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Advantages

(i) Best for binary classification
(ii) Works well with Sigmoid output.
(iii) Faster convergence than MSE in classification

Disadvantages

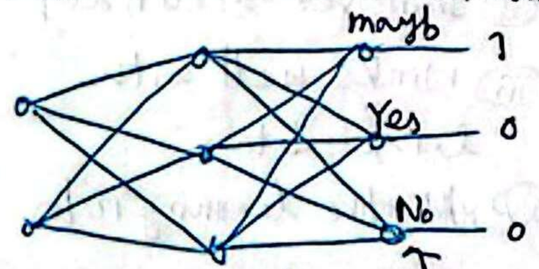
(i) Requires output (0-1)
(ii) Sensitive to extreme probabilities
(iii) Not for multi class problem

CGPA	IQ	Jobs
3	3	0
1	2	0
4	10	1

Categorical Cross Entropy:

is a loss function used in multi-class classification problems in machine learning and deep learning. It measures the difference between the true class distribution and predicted probability distribution produced by a model. It is commonly used with the softmax function in neural networks.

CGPA	IQ	Job
3	3	maybe
2	2	No
4	10	Yes



$$\text{loss} = -\sum_{j=1}^{k} y_j \log(\hat{y}_j)$$

$$\text{cost} = \frac{-1}{n} \sum_{i=1}^{n} \sum_{j=1}^{k} y_j \log(\hat{y}_j)$$

Advantages

(i) Best for multi-class classification
(ii) Works with Softmax

Disadvantages

(i) Requires one-hot encoding.
(ii) Computationally heavier than Binary cross Entropy.

Optimizer:

An optimizer is an algorithm used to update the models weights and biases in order to minimize the loss function during training.

Advantages

(i) Faster convergence.
(ii) Improves accuracy.
(iii) Works well with large data.
(iv) Adaptive learning rates.

Disadvantages

(i) May get stuck in local minima.
(ii) sensitive to learning rate.
(iii) Higher computational costs
(iv) Requires hyperparameter tuning.

Gradient Descent:

Is an optimization algorithm but instead of using individual training example like SGD. it calculates the average gradient of the cost function of entire dataset

Advantages

(i) Stable and smoth convergence.
(ii) Accurate gradient calculation.
(iii) Good for small datasets.

Disadvantages

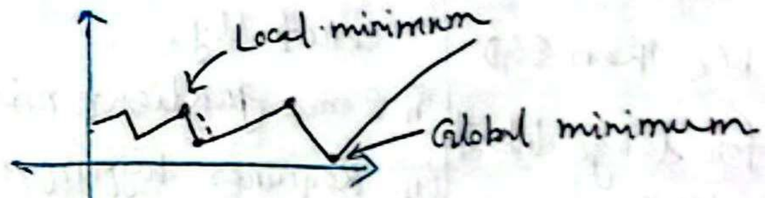
(i) Very slow for large dataset.
(ii) High memory usage
(iii) Cannot be used easily for very big data

Batch gradient descent = Gradient descent.

Stochastic Gradient Descent:

Here, the models updates weights using one training example at a time.

* If dataset has 10000 samples → 1000 updates per epoch.



Advantages

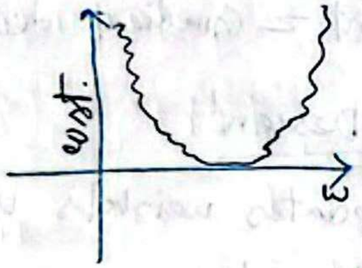
(i) Much faster than Batch GD.
(ii) Can escape local minima.
(iii) Works well for large dataset.
(iv) Low memory usage.

Disadvantages

(i) very noisy updates.
(ii) Loss fluctuates heavily.
(iii) May never perfectly reach minimum.

Mini-Batch Stochartic Gradient Descent:

is an optimization technique where the training dataset is devided into small batches and the model updates its weights after processing each batch instead of the whole dataset on a single sample.



Advantages

(i) Faster training.
(ii) More stable than SGD.
(iii) Efficient for large datasets.
(iv) Supports parallel processing.

Disadvantages

(i) Batch size much be chosen carefully.
(ii) Some gradient noise
(iii) Requires hyperparameter tuning.
(iv) Can get stuck in local minima.

Removing noise:

SGD with momentum → [Exponential weight Avg.]

$$w_{new} = w_{old} - \eta \frac{\delta \mathcal{L}}{\delta w_{old}}$$

$$b_{new} = b_{old} - \eta \frac{\delta \mathcal{L}}{\delta b_{old}}$$

Exponential Weight Average:

$t_1 \quad t_2 \quad t_3 \quad -- \quad t_n$

$a_1 \quad a_2 \quad a_3 \quad -- \quad a_n$

$$w_t = w_{t-1} - \eta \frac{\delta \mathcal{L}}{\delta w_{t-1}}$$

$$V_{t_1} = a_1$$

$$V_{t_2} = \beta V_{t_1} + (1-\beta) a_2$$

$$V_{t_3} = \beta t_2 + (1-\beta) a_3$$

$$w_t = w_{t-1} - \eta V_{dw}$$

$$V_{dwt} = \beta V_{t-1} + (1-\beta) \frac{\delta \mathcal{L}}{\delta w_{t-1}}$$

Adagrad - Adaptive Gradient Descent:

is an ~~algorithm~~ optimization algorithm that adapts the learning rate for each parameter by dividing the learning rate by the square root of the accumulated past squared gradients.

$$w_t = w_{t-1} - \eta \frac{\delta \mathcal{L}}{\delta w_{t-1}}$$

$$\downarrow$$

$$w_t = w_{t-1} - \eta' \frac{\delta \mathcal{L}}{\delta w_{t-1}}$$

here, $\eta' = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}$ ← small value

$$= \frac{\eta}{\sum_{i=1}^{t} \left(\frac{\delta \mathcal{L}}{\delta w_i}\right)}$$

RMS Prop:

RMS prop improves training by adapting the learning rate using a moving average of squared gradients, preventing the learning rate from becoming too small and allowing faster and more stable convergence.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g]_t + \epsilon}} \cdot g_t$$

Adagrad and RMS prop:

$$\eta' = \frac{\eta}{\sqrt{S_{dw} + \epsilon}}$$

→ exponential weight average

$$Sdw_t = \beta * Sdw_{t-1} + (1-\beta)\left(\frac{\delta w}{\delta w_{t-1}}\right)$$

Momentum + RMS prop: (Adam Optimizer)

V_{dw} V_{db} Sdw Sdb

$$V_t = W_{t-1} - \eta' V_{dw} \longrightarrow \eta' = \frac{\eta}{\sqrt{Sdw + \epsilon}}$$

$$V_{dw} = \beta * V_{dw_{t-1}} + (1-\beta)\frac{\delta L}{\delta W_{t-1}}$$

$$b_t = b_{t-1} - \eta' V_{db} \longrightarrow V_{db} = \beta * V_{db_{t-1}} + (1-\beta)\frac{\delta L}{\delta b_{t-1}}$$

$$\eta' = \frac{\eta}{\sqrt{Sdb + \epsilon}}$$

Linear Regression:

Hidden layer → ReLU
Output layer → Linear Activation
Loss function → MSE, MAE, Huber.

Classification:

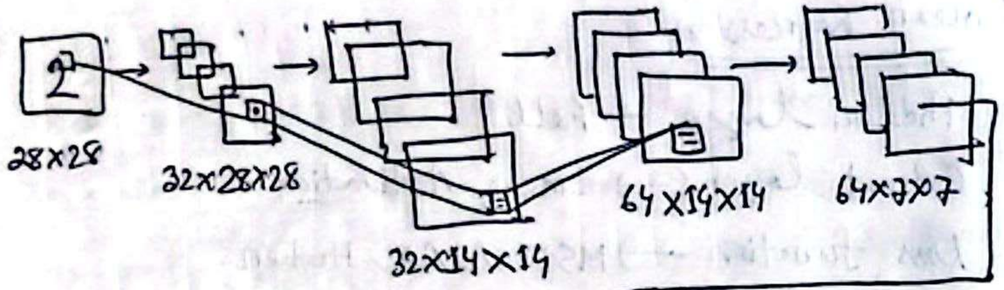
Binary
H.L → ReLU
O.L → Sigmoid
L.F → Binary Cross Entropy

Multi class
H.L → ReLU
O.L → Softmax
L.F → Categorical Cross Entropy.

CNN

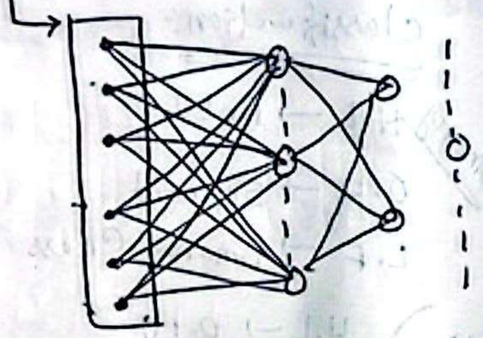
Convolutional Neural Network is a deep learning model that uses convolution operations to automatically extract features from images and perform tasks such as classification and detection.

- Kernal (Filter, feature detection)
- Stride
- Padding
- Pooling
- Flatten.



Convolution
padding = 1
Kernal = 3x3
Stride = 1
+
ReLU

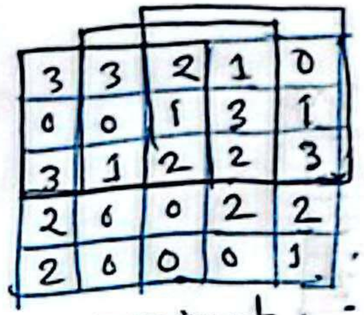
Max pooling
Kernal = 2x2
Stride = 2



Flatten

Kernal!

→ is nothing but a filter that used to extract the feature from the images.
→ Kernel is a matrix that moves over the input data, performs the dot product with the sub region of input data and gets the output as the matrix of dot products.
→ Kernal moves on the input data by stride value.



5x5 input

Kernel 3x3

0	1	2
2	2	0
0	1	2

3x0 +3x1 +2x2 +0x2
+0x2 +1x0 +3x0 +
1x1 +2x2

0+3+4
+0+0+0
+0+1+4

12	12	17
10	17	19
9	6	14

$O = [i - K] + 1$

$= [5 - 3] + 1$

$= 3$

Stride!

→ The filter is moved across the image left to right, top to bottom with an one-pixel column change on the horizontal movements, then an one-pixel row change on the vertical movements.
→ The amount of movement between applications of the filter to the input image is refered to as the stride and it is almost always symmetrical in height and width dimensions.
→ The default stride or strides in two dimensions is (1,1) for the height and width movement.

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

5x5 input

Kernal 3x3

0	1	2
2	2	0
0	1	2

12	12	17
10	17	19
9	6	14

12	17
9	14

$$O = \left[\frac{i-K}{S}\right] + 1$$

$$= \left[\frac{5-3}{2}\right] + 1$$

$$= 2$$

Padding:

→ Padding is the best approach where the number of pixels needed for the convolutional Kernal to process the edge pixels are added into the outside copying the pixels from the edge of the image.

→ Fix the border effect problem with padding.

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0	1	3	1	0
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	6	0	0	0	0	0

0	1	2
2	2	0
0	1	2

$$O = \left[\frac{i-k+2p}{s}\right] + 1$$

$$= \left[\frac{5-3+2\times1}{1}\right] + 1$$

$$= 5 \rightarrow 5\times5$$

6	14	17	11	3
14	12	12	17	11
8	10	17	19	13
11	9	6	14	12
6	4	4	6	4

Pooling:

→ Pooling is required to down sample the detection of features in feature map.

→ Pooling layers provide an approach to down sampling feature map by summerizing the presence of features in patches of the feature map. There are two common methods (Average and max pooling).

6	14	17	13	3
14	12	12	17	11
8	10	7	19	13
11	9	6	14	12
6	4	4	6	4

14	17
10	19

Flattening:

Once the pooled feature map is obtained the next step is to flatteren it. It involves transforming the entire pooled feature map matrix into a single column which is then fed to the neural network for processing.

Data Preprocessing:

* **Column normalization:** means scaling the values of a feature so that they fall within a specific range. usually :[0, 1]

Most common method :- ~~Mean~~ Min-Max scaling.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

* **Column Standardization:** transforms data so that mean = 0, and standard deviation = 1

$$x' = \frac{x - \mu}{\sigma}$$ Here, μ = mean

σ = standard deviation

$$= \sqrt{\frac{\Sigma (x-\mu)^2}{n}}$$

Dimention reduction:

- PCA (Principle Component Analysis)
- t-SNE (t- Distributed Stochastic Neighbor Embedding)

PCA:

Reduce high dimentional data into fewer dimentions while preserving maximum information.

Advantages

(i) Reduce dimentionality
(ii) Speeds up training
(iii) Usefull for visualization.
(iv) Reduces noise.

Disadvantages

(i) Information lost.
(ii) Difficult to interpret component.
(iii) Works mainly for linear relationship.
(iv) Sensitive to scaling.

t-SNE :-

is a non-linear dimentionality reduction technique mainly used for visualization - It preserves local structure

Advantages

(i) Visualizing high dimentional embedding.
(ii) Deep learning feature visualization.
(iii) Word/image embedding.

Disadvantages

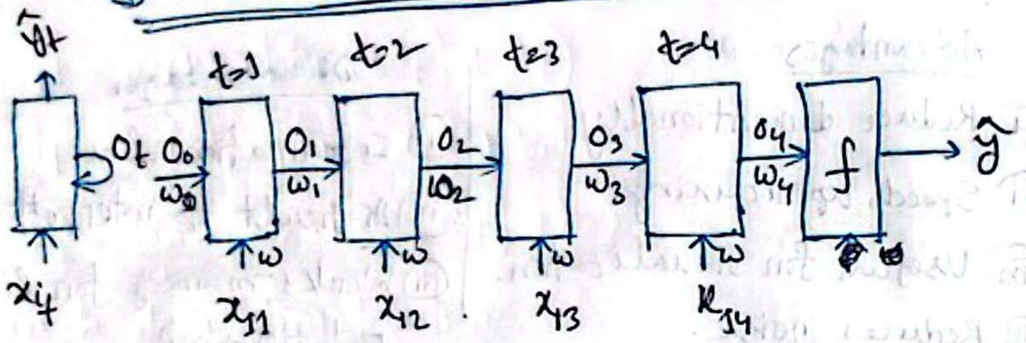
(i) Very slow
(ii) Cannot used for large dataset easily.
(iii) Not good for new unseen data

ANN

Artificial Neural Network is a computational model in ML and AI that inspired by the structure and functioning of the human brain.

Def: ANN is a computing system made of layers of interconnected neurons that learn from data by recognize patterns and make decisions.

Reaurrent Neural Network (RNN)



<I Love Bangladesh>

$O_1 = f(x_{11}\omega + O_0\omega_0)$ $\quad$ loss = $y - \hat{y}$

$O_4 = f(x_{14}\omega + O_3\omega_3)$

$\omega''_{new} = \omega''_{old} - \eta \frac{\delta L}{\delta \omega''_{old}}$

$$\frac{\delta L}{\delta \omega''_{old}} = \frac{\delta \hat{y}}{\delta O_4} \times \frac{\delta O_4}{\delta O_3} \times \frac{\delta O_3}{\delta O_2} \times \frac{\delta O_2}{\delta \omega''_{old}}$$

~~LSTM (Long Short Term Memory):~~

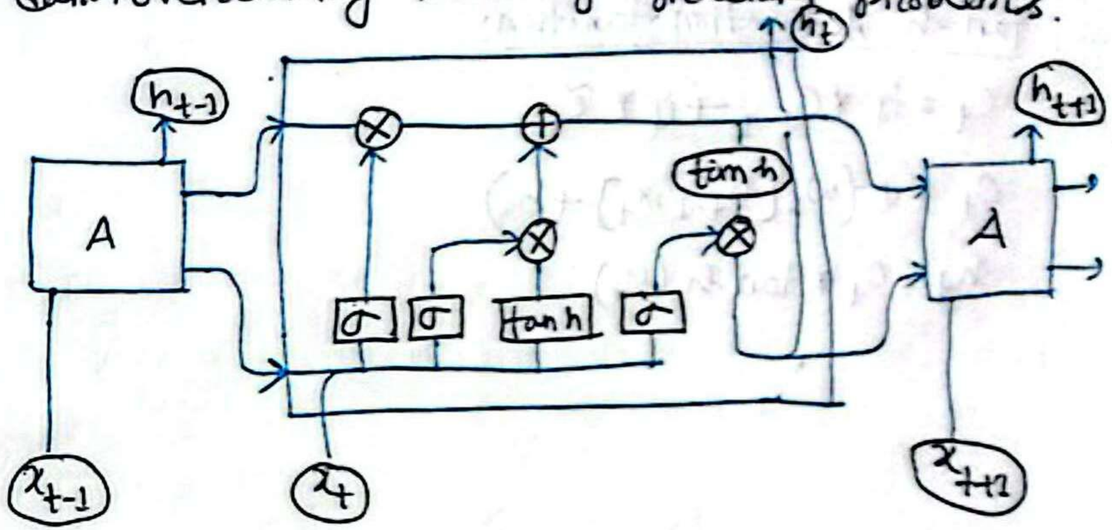
~~LSTM models are specialized RNN~~

Definition: A RNN is a neural network where the output from the previous step is fed back into the network as input, allowing it retain memory of past information.

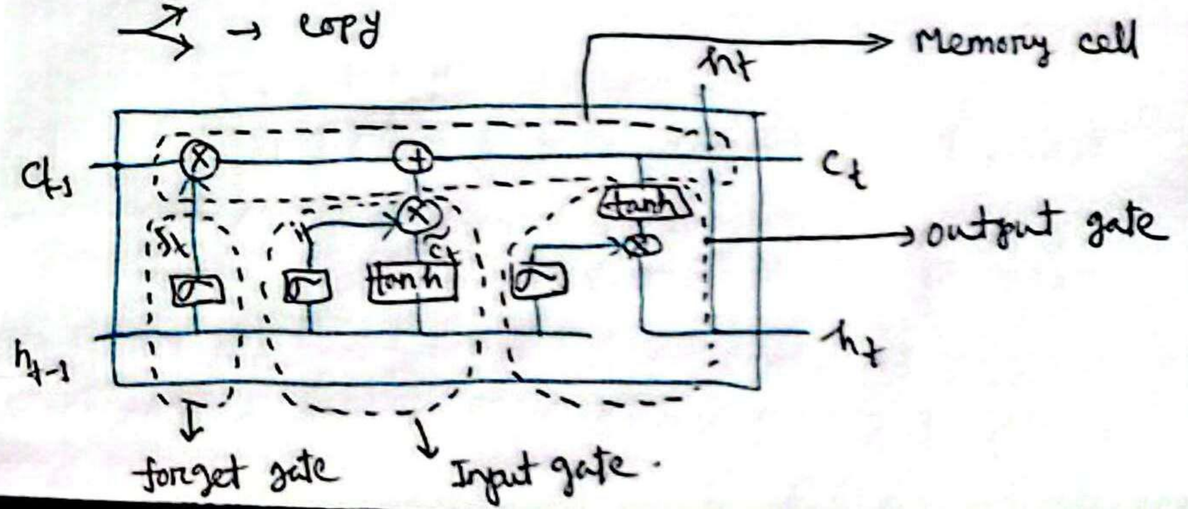
Example uses: (i) Language translation
(ii) Speech recognition
(iii) Text prediction
(iv) Time series forecasting.

LSTM (Long Short-term Memory)

LSTM models are specialized RNN architectures designed to long-term dependencies in sequential data, overcoming vanishing gradient problems.



⊗ → pointwise operation
⊕ → addition
→ → vector transfer
→ → concatenate
→ → copy



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

tan h Activation function:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$O_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = O_t * \tanh(C_t)$$